

Hierarchical JSSP-Based Scheduling With Smooth Trajectory Optimization for Autonomous Intersection Management

Author Name

Abstract—Autonomous intersection management can improve traffic efficiency when connected autonomous vehicles communicate through V2V and V2X channels, but a schedule that is optimal at an abstract conflict-zone level is not automatically executable by real vehicles. This paper proposes a hierarchical framework that formulates intersection passing order as a job shop scheduling problem and learns a high-level scheduler with reinforcement learning. The scheduled conflict-zone entry times are then passed to a low-level trajectory smoother that adjusts vehicle speed in advance, reducing unnecessary stops before the intersection while preserving safety constraints. The framework is designed for event-triggered replanning in SUMO and future vehicle-in-the-loop validation in CARLA. The expected outcome is an intersection manager that combines the global ordering strength of graph-based JSSP scheduling with smoother, more energy-aware vehicle motion.

Index Terms—Autonomous intersection management, connected autonomous vehicles, job shop scheduling problem, reinforcement learning, trajectory optimization, velocity smoothing, V2X.

I. INTRODUCTION

AUTONOMOUS vehicles are increasingly deployed in structured environments such as intelligent factories, warehouses, and ports, while connected autonomous vehicles are also becoming more relevant for urban traffic systems. With vehicle-to-vehicle and vehicle-to-infrastructure communication, an intersection no longer needs to be controlled only by fixed signal phases or human driving conventions. Instead, autonomous intersection management can coordinate the passing order of vehicles before they reach the conflict area.

Early scheduling methods often rely on predefined rules such as first-come-first-served policies, reservation mechanisms, or hand-designed heuristics. These methods are intuitive and easy to deploy, but their search space grows rapidly when traffic density increases and many vehicles compete for overlapping conflict zones. More recent graph-based and job-shop-scheduling-inspired formulations provide a compact way to represent the conflict relationship between vehicles and intersection zones.

However, a high-quality abstract schedule is only one part of the control problem. After a vehicle receives an assigned passing order or scheduled conflict-zone time, it must still adapt its speed and acceleration to satisfy the schedule. If

vehicles simply stop at the stop line and wait for their turn, the intersection may suffer from stop-and-go waves, longer delay, and higher energy consumption. A smoother velocity profile can reduce unnecessary stops and make the schedule more practical for real vehicle execution.

This paper aims to bridge this gap by combining high-level JSSP-based scheduling and low-level trajectory smoothing. The proposed framework recalculates the schedule when vehicles enter or leave the intersection management region, and then uses the scheduled times as references for speed optimization. The high-level scheduler is intended to be trained in SUMO through reinforcement learning, while the low-level smoother prepares the method for later vehicle-in-the-loop validation in CARLA.

The main contributions are summarized as follows:

- A hierarchical autonomous intersection management framework that connects JSSP-based passing-order optimization with vehicle-level trajectory execution.
- An event-triggered graph scheduling mechanism that updates the intersection schedule when the active vehicle set changes.
- A low-level speed smoothing formulation that tracks scheduled conflict-zone times while reducing unnecessary stop-line waiting.
- A simulation-oriented evaluation plan comparing the proposed method with first-come-first-served and stop-at-intersection baselines.

II. RELATED WORK

A. Autonomous Intersection Management

Autonomous intersection management replaces fixed traffic signals with negotiated or optimized vehicle movements and has been widely discussed as a promising direction for connected autonomous traffic systems [1]. Reservation-based approaches divide the intersection into spatial or temporal resources, while rule-based methods determine priority through arrival time, lane, or safety margins. These methods are practical baselines because they are interpretable and computationally light, but they may become conservative when traffic demand is high.

B. Scheduling and Graph-Based Formulations

The job shop scheduling problem provides a natural abstraction for intersection coordination. Each vehicle can be

Author information, affiliation, and funding acknowledgment will be updated before submission.

Manuscript prepared as an IEEE journal-style draft.

treated as a job, each conflict zone as a machine, and each vehicle-zone traversal as an operation. Precedence constraints preserve the geometric order of zones along a path, while disjunctive constraints prevent two conflicting vehicles from occupying the same zone at the same time. This abstraction is attractive because it converts a traffic coordination problem into a structured combinatorial optimization problem.

C. Reinforcement Learning for Combinatorial Scheduling

Reinforcement learning has been used to learn dispatching policies for combinatorial scheduling problems where hand-crafted priority rules are difficult to tune. In a JSSP setting, an agent can observe the current partial schedule and select the next schedulable operation. For intersection management, this mechanism can be adapted to select which vehicle should enter a conflict zone next. The key challenge is to design an observation representation that captures safety, route order, and traffic density while remaining computationally efficient.

D. Trajectory Smoothing and Stop Avoidance

Even when the passing order is optimized, vehicles still need feasible speed trajectories to execute the schedule. Stop-line waiting is simple but may increase delay, acceleration variation, and energy consumption. Smooth trajectory planning methods adjust vehicle speed before reaching the intersection so that vehicles can arrive at conflict zones close to their assigned time. This paper follows this direction and treats low-level smoothing as an execution layer below the learned scheduler.

III. PROBLEM FORMULATION

A. Intersection Conflict-Zone Model

Consider an intersection management region with a set of connected autonomous vehicles $\mathcal{V}$ and a set of conflict zones $\mathcal{Z}$. Each vehicle $i \in \mathcal{V}$ follows a known route through an ordered conflict-zone sequence

$$\mathcal{R}_i = (z_{i,1}, z_{i,2}, \dots, z_{i,n_i}), \quad z_{i,k} \in \mathcal{Z}. \quad (1)$$

The route sequence is determined by the vehicle's incoming lane, outgoing lane, and turning movement.

B. JSSP Representation

The intersection scheduling problem is formulated as a job shop scheduling problem. Vehicle i is treated as a job J_i , conflict zone z is treated as a machine M_z , and the traversal of vehicle i through zone $z_{i,k}$ is an operation $O_{i,k}$. Each operation has a processing time $p_{i,k}$ that approximates the time required for vehicle i to occupy and clear conflict zone $z_{i,k}$.

Let $s_{i,k}$ denote the scheduled start time of operation $O_{i,k}$. The route order imposes the precedence constraint

$$s_{i,k+1} \geq s_{i,k} + p_{i,k}, \quad k = 1, \dots, n_i - 1. \quad (2)$$

If two operations $O_{i,k}$ and $O_{j,l}$ use the same conflict zone, they cannot overlap:

$$[s_{i,k}, s_{i,k} + p_{i,k}) \cap [s_{j,l}, s_{j,l} + p_{j,l}) = \emptyset. \quad (3)$$

C. Objective

The high-level scheduler seeks a feasible ordering of conflict-zone operations that reduces traffic inefficiency. A general objective can be written as

$$\min_{\{s_{i,k}\}} \alpha C_{\max} + \beta \sum_{i \in \mathcal{V}} d_i + \gamma \sum_{i \in \mathcal{V}} n_i^{\text{stop}}, \quad (4)$$

where $C_{\max}$ is the schedule makespan, d_i is vehicle delay, n_i^{stop} is the number of stops, and α , β , and γ are weights selected for the study.

D. Safety and Communication Assumptions

The first draft assumes that each vehicle communicates its route, position, speed, and acceleration to the intersection manager after entering the control region. Vehicle dynamics are bounded by speed and acceleration limits. If communication is lost or a trajectory becomes infeasible, the vehicle falls back to conservative stop-line behavior.

IV. HIERARCHICAL INTERSECTION MANAGEMENT FRAMEWORK

The proposed framework contains two coupled layers. The high-level layer builds a JSSP graph from the active vehicle set and computes the conflict-zone schedule. The low-level layer receives scheduled conflict-zone entry times and generates smooth vehicle speed commands that track the schedule while satisfying physical constraints.

A. Event-Triggered Replanning

The scheduler is updated when a vehicle enters the management region, leaves the intersection, or violates a schedule-tracking tolerance. These events change the feasible operation set or the expected occupancy time of conflict zones. Between events, the low-level smoother continuously adjusts vehicle speed without forcing a full graph rescheduling step.

B. Information Flow

At each scheduling event, the manager receives the active vehicle states, route sequences, and estimated processing times. It outputs a schedule containing operation order and target start times. Each vehicle then solves a local speed-smoothing problem using its current state, route geometry, and assigned conflict-zone times.

V. REINFORCEMENT-LEARNING-BASED JSSP SCHEDULER

A. Graph State Representation

The scheduler represents the current intersection as a graph whose nodes are operations and whose edges encode precedence and conflict relationships. Precedence edges connect consecutive conflict zones on the same vehicle route. Conflict edges connect operations that require the same conflict zone. At each decision step, only operations whose predecessors have already been scheduled are schedulable.

Require: Active vehicles $\mathcal{V}$, conflict zones $\mathcal{Z}$, current time t
Ensure: Scheduled conflict-zone times and smoothed speed references

- 1: Detect whether a vehicle enters, exits, or violates tracking tolerance.
- 2: **if** a scheduling event is triggered **then**
- 3: Build the JSSP graph from $\mathcal{V}$ and vehicle route sequences.
- 4: Select operation ordering using the learned RL scheduler.
- 5: Convert operation order into target conflict-zone entry times.
- 6: **end if**
- 7: **for** each active vehicle $i \in \mathcal{V}$ **do**
- 8: Generate a smooth velocity trajectory that tracks the scheduled times.
- 9: Apply safety constraints and conservative fallback if needed.
- 10: **end for**

Fig. 1. Event-triggered hierarchical intersection management.

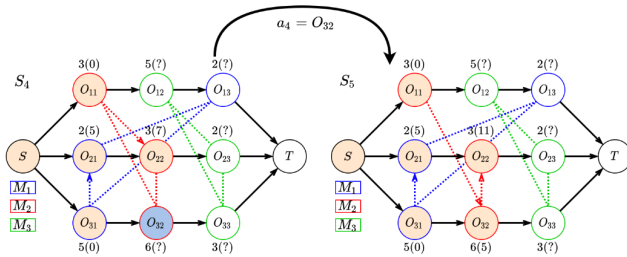


Figure 2: Example of state transition. Orange nodes are operations already scheduled, and the grey node is operation selected to be scheduled at current state. Integers in bracket are starting time of scheduled operations, where unscheduled operations have unknown starting time (denoted as ?).

Fig. 2. Draft reference for a JSSP/RL state-transition idea. TODO: redraw this figure in an original style and cite the source paper before submission.

B. MDP Definition

The Markov decision process is defined by four elements:

- **Observation:** graph features including operation status, estimated processing time, vehicle arrival time, route order, and conflict-zone occupancy.
- **Action:** selection of one schedulable operation, equivalent to assigning priority to a vehicle-zone traversal.
- **Transition:** update of the partial schedule after the selected operation is placed, including newly schedulable operations.
- **Reward:** negative schedule cost, such as incremental delay, makespan increase, stop penalty, or a weighted combination of these terms.

C. Training in SUMO

Training episodes are generated in SUMO by sampling traffic demand, vehicle routes, and arrival patterns. The RL scheduler observes the graph state and constructs a passing order for active vehicles. The simulation returns delay, waiting time, and schedule feasibility signals that are converted into rewards. After training, the learned policy is evaluated under unseen traffic densities and intersection layouts.

VI. LOW-LEVEL TRAJECTORY SMOOTHING

The low-level controller converts scheduled conflict-zone entry times into feasible speed commands. Instead of stopping at the intersection and waiting for permission, a vehicle begins adjusting its speed upstream so that it reaches each conflict zone near the assigned time.

A. Scheduled-Time Tracking

For vehicle i , let $\hat{t}_{i,k}$ denote the predicted arrival time at conflict zone $z_{i,k}$ under the current velocity plan. The tracking error is

$$e_{i,k} = \hat{t}_{i,k} - s_{i,k}. \quad (5)$$

The smoother penalizes large tracking errors while also discouraging excessive acceleration and jerk.

B. Smooth Velocity Optimization

A generic smoothing objective can be written as

$$\min_{\mathbf{u}_i} \sum_k w_t e_{i,k}^2 + \sum_h w_a a_i(h)^2 + \sum_h w_j j_i(h)^2, \quad (6)$$

subject to speed, acceleration, safety-distance, and conflict-zone time-window constraints. Here $\mathbf{u}_i$ denotes the control sequence, $a_i(h)$ is acceleration, and $j_i(h)$ is jerk at prediction step h .

C. Barrier and Fallback Conditions

Barrier-function constraints can be used to keep vehicles outside occupied conflict zones and preserve longitudinal safety. If the scheduled-time tracking problem becomes infeasible, the controller switches to a conservative fallback that stops before the intersection and requests a new high-level schedule.

VII. EXPERIMENTAL SETUP

A. Simulation Platform

The first evaluation stage will be conducted in SUMO because it supports scalable microscopic traffic simulation and repeated training episodes. A later validation stage can connect the scheduler and smoother to CARLA to examine vehicle-level execution under richer dynamics and perception assumptions.

TABLE I
CANDIDATE BASELINE METHODS

Method	Description
FCFS stop-line	Vehicles are served by arrival order and wait before entering the intersection.
Heuristic scheduler	Passing priority is selected by a handcrafted rule such as earliest arrival or shortest traversal time.
JSSP scheduler only	High-level schedule is used without low-level velocity smoothing.
Proposed hierarchy	JSSP/RL scheduler plus low-level smooth trajectory tracking.

TABLE II
CANDIDATE EVALUATION METRICS

Metric	Purpose
Average delay	Measures traffic efficiency.
Throughput	Counts vehicles that clear the intersection per unit time.
Number of stops	Evaluates stop-and-go reduction.
Acceleration variance	Measures trajectory smoothness.
Energy proxy	Estimates energy-related cost from speed and acceleration profiles.
Computation time	Checks real-time scheduling feasibility.

B. Scenarios and Baselines

The experiments will include multiple traffic densities and route distributions. The primary baseline is a first-come-first-served controller that requires vehicles to stop at the intersection stop line before crossing. Additional baselines may include heuristic priority rules and a scheduler without trajectory smoothing.

C. Evaluation Metrics

VIII. RESULTS AND DISCUSSION

A. Scheduling Performance

This subsection will report whether the learned JSSP scheduler improves intersection throughput and reduces delay under different traffic densities. The analysis should include both average performance and variability across random seeds.

B. Trajectory Smoothness

This subsection will evaluate whether the low-level smoother reduces unnecessary stops before the intersection. Useful plots include speed-time profiles, acceleration-time profiles, and histograms of stop counts.

C. Ablation Study

An ablation study should separate the contribution of high-level scheduling from that of low-level trajectory smoothing. The expected comparison is between the proposed hierarchy, the JSSP scheduler without smoothing, and the FCFS stop-line baseline.

D. Limitations

The current framework assumes reliable communication and known routes inside the management region. Future work should test robustness to delayed messages, uncertain vehicle dynamics, mixed traffic with human-driven vehicles, and deployment on vehicle-in-the-loop platforms.

IX. CONCLUSION

This paper draft presents a hierarchical framework for autonomous intersection management that combines JSSP-based high-level scheduling with low-level smooth trajectory optimization. The core idea is to use reinforcement learning to construct efficient conflict-zone schedules while enabling vehicles to execute those schedules through advance speed adjustment rather than stop-line waiting. The planned evaluation will compare the proposed method against FCFS and heuristic baselines in SUMO, with future validation in CARLA. The long-term goal is to make graph-based intersection scheduling more executable for real connected autonomous vehicles.

ACKNOWLEDGMENT

The author would like to thank collaborators and supervisors who provide feedback on the intersection management formulation, simulation design, and trajectory smoothing method.

REFERENCES

- [1] Z. Zhong, M. Nejad, and E. E. Lee, "Autonomous and semi-autonomous intersection management: A survey," *arXiv preprint arXiv:2006.13133*, 2020.